Lists and Tuples

~ G. Anushiya Rachel
Assistant Professor, CSE
Shiv Nadar University Chennai

Introduction to Lists

- Sequence of values, called elements or items, which need not be of the same type, and are enclosed within square brackets
- Lists are mutable and have indices similar to strings.
  - Mutable – elements can be altered

Assigning a List

Examples:

- cheese = [‘Cheddar’, ‘Mozzarella’, ‘Gouda’]
- numbers = [12, 14, 35, 67]
- empty = []
- variety = [‘spam’, 2.0, 5, [10,20]]

Indexing and Slicing

- To access an element in a list, use indices.

$$l = [1, 2,' 3',' q','e']$$
$$print(l[0], l[-2])$$

  - $l[0]$ would refer to 1 and $s[-2]$ would refer to $'q'$
- Lists are immutable, therefore, equating $l[i]$ to another character or number is supported.
  - $l[2] = 'r'$ # Supported
  - Slicing – used to obtain a subset of the list

$$l = ['h',' e',' l',' 0', 1, 2]$$
$$print(l[3:5])$$

  - This will result in $['0', 1]$

Slicing

- Let $l = [1, 2, 3, 4, 'a', 'b', 'c', 'd']$
- $l[:]$ – Entire list
  - $[1, 2, 3, 4, 'a', 'b', 'c', 'd']$
- $l[1:]$ – Elements from index 1 to end
  - $[2, 3, 4, 'a', 'b', 'c', 'd']$
- $l[:3]$ – Elements from index 0 to 2
  - $[1, 2, 3]$
- $l[:5:2]$ – Elements from 0 to 4 in steps of 2 (alternate elements)
  - $[1, 3, 'a']$
- $l[::-1]$ – Elements from the end to the beginning – Reverse of the list
  - $['d', 'c', 'b', 'a', 4, 3, 2, 1]$

List Operations

- List concatenation (+):
  - $l1 = [1, 2, 3]$
  - $l2 = [4, 5, 6]$
  - $print(l1 + l2)$ # Prints [1,2,3,4,5,6]

- List multiplication by an integer (∗):
  - $print(\ [1,2,3] * 3)$ # Prints [1,2,3,1,2,3,1,2,3]

- List comparison using relational operators:
  - $l1 == l2$ # $False$
  - $l1 < l2$ # $True$

List Operations

- List membership test using in:
  - $print('a' in ['a','b','c'])$ # $True$
  - $print('f' in [1,'b','c'])$ # $False$

- Iterating through a list:
  - $for\ i\ in\ l1$:
    $print(i)$ # Prints each element in the list

  (OR)
  - $for\ i\ in\ range(len(l1))$:
    $print(l1[i])$

Cloning and Aliasing

- Consider the following statements:

  $>>> l1 = [1, 2, 3, 4]$

  $>>> l2 = l1$ → Aliasing – providing a new name for the same list, an alias

  $>>> l3 = l1[:]$ → Cloning – creating an independent copy

  $>>> l2[3] = 5$

  $>>> print(l1, l2, l3)$

- Result of the above statements:
  - [1,2,3,5] [1,2,3,5] [1,2,3,4]
  - Changes in $l2$ affect $l1$ and vice-versa.
  - However, changes in $l1$ and $l3$ do not affect each other.

List Methods – append, extend

- Adding elements to a list using ***append***

$$>>> L = [1, 2]$$

$$>>> L.append(5)$$

$$>>> print(L)$$

$$L = [1, 2, 5]$$

- Adding elements using ***extend*** (input must be a sequence – list, tuple, string)

$$>>> L1 = [‘a’, ‘b’]$$

$$>>> L.extend(L1)$$

$$>>> print(L)$$

$$[1, 2, 5, ‘a’, ‘b’]$$

List Methods – sort, reverse

- Sorting the elements of a list using ***sort***

```
>>> L = ['eat','more','SPAM!','please']
>>> L.sort()                # Elements must be of the same data type
>>> L
['SPAM!','eat','more','please']
>>> L1 = ['abc','aBe','ABD']
>>> L1.sort(key = str.lower)
>>> L1
['abc','ABD','aBe']
```

Sort in descending order:

```
L1.sort(reverse = True)
```

Sort based on length:

```
L1.sort(key = len)
```

- Reverse the order using ***reverse***

```
>>> L1.reverse()
```

List Methods – pop, count, index

- Deleting the last element in a list and displaying it using ***pop***

```
>>> L = [1, 2, 5, 9]
>>> L.pop()
9
```

To remove an element at a specific index, use L.pop(index)

- Counting the number of occurrences of a value using ***count***

```
>>> L.count(2)
1
```

- Return the index of an element using ***index***

```
>>> L = ['spam','eggs','ham']
>>> L.index('eggs')
1
```

List Methods – insert, remove, clear

- Insert element at specified position using ***insert***

  >>> $L.insert(1, 'toast')$

  >>> L

  ['spam', 'toast', 'eggs', 'ham']

- Delete a value using ***remove*** – deletes the first occurrence

  >>> $L.remove('eggs')$

  >>> L

  ['spam', 'toast', 'ham']

- Remove all elements of a list using ***clear***

  >>> $L.clear()$

  []

List Functions – len, list

- Length of a list using ***len***

```
>>> L = [1, 2, 3, 4]
>>> len(L)
4
```

- Conversion to list using ***list***

```
>>> word = 'python'
>>> list(word)
['p', 'y', 't', 'h', 'o', 'n']
```

List Functions – del, map

- Deleting an element in a list using ***del***

```
>>> del(L[0])
>>> print(L)
[2, 10, 'a', 'b']
```

- ***map*** – Applies a function to items in a sequence

```
>>> list(map(abs, [1, -2, -4]))
[1, 2, 4]
>>> import math
>>> list(map(math.sqrt, [36, 4, 144]))
[6.0, 2.0, 12.0]
```

Programming Exercises

1. Consider a list of N numbers. Determine if there exists an element in the list such that the sum of the elements on its left is equal to the sum of the elements on its right. If there are no elements to the left/right, then the sum is considered to be zero.
2. Sunny and Johnny together have M dollars they want to spend on ice cream. The parlor offers N flavors, and they want to choose two flavors so that they end up spending the whole amount. You are given the cost of these flavors. You have to display the indices of the two flavors whose sum is M.
3. Given a list of integer values, find the fraction of count of positive numbers, negative numbers and zeroes to the total numbers. Print the value of the fractions correct to 3 decimal places.

List Comprehension

- Consider the following:

```
>>> res = []
>>> for i in 'SPAM':
        res.append(i*4)
>>> res
['SSSS'. 'PPPP', 'AAAA', 'MMMM']
```

- This for loop can be written in short as

```
>>> res = [i * 4 for i in 'SPAM']
```

List Comprehension

- Example:

```
>>> numbers = []
>>> for i in range(1,5):
        numbers.append(i**2)
>>> numbers
[1, 4, 9, 16]
```

- This for loop can be written in short as

```
>>> numbers = [i**2 for i in range(1,5)]
```

Nested Lists

- Lists within lists
- Examples:
  - $L1 = ['a','b',['cc','dd',['eee','fff']],'g','h']$
  - $L2 = [[1,2,3],[4,5,6],[7,8,9]]$
- Accessing elements:
  - Examples:
    - To access 'eee' from $L1$ – $L1[2][2][0]$
    - To access 5 from $L2$ – $L2[1][1]$
    - To access 7 and 8 from $L2$ – $L2[2][0:2]$
    - To access 7, 8, and 9 from $L2$ – $L2[2]$

Nested Lists

- Add items to a nested list:

  $>>> L = ['a', ['bb', 'cc'], 'd']$

  $>>> L[1].append('xx')$

  $>>> L$

  $['a', ['bb', 'cc', 'xx'], 'd']$

  - $insert, extend$ methods can be used as well.

- Similarly, all other list methods and functions may be used on nested lists as well.

Programming Exercises

4. Find items starting with a specific character (obtained from the user) from a given list.
5. Remove consecutive duplicates from a given list. (E.g.: [1, 2, 2, 3, 4, 2] → [1, 2, 3, 4, 2])
6. Create a run-length encoded list from a given list. (E.g.: [1, 1, 1, 2, 1, 2, 2, 3] → [[1,3], 2, 1, [2, 2], 3]
7. Get a nested list from a user and sort the list based on the last element of each list.

   Sample input: [[1,2], [3, 4, 5], [1, 1], [3, 4, 9]]

   Expected output: [[1,1], [1, 2], [3, 4, 5], [3, 4, 9]]
8. Create a 3x4x2 dimensional array, where each element is '*'.

Programming Exercises

9. Get two 2D lists from a user and add them.
   Sample input: [[1, 2], [2,4], [3,5]] and [[3, 1], [1, 1], [2, 3]]
   Expected output: [[4, 3], [3, 5], [5, 8]]
10. Find the list in a list of lists whose sum of elements is the highest.
11. Remove duplicates from a list of lists.
   Sample input: [[10, 20], [40], [30, 56, 25], [10, 20], [33], [40]]
   Expected output: [[10, 20], [40], [30, 56, 25], [33]]
12. Get a list of strings from a user and rotate each string N times and return the resulting list.
13. Obtain N lists from a user and return the list of indices at which all N lists contain the same element.

Programming Exercises

14. Read a string from the user containing a date in the form mm/dd/yyyy and print the date in the form March 23, 2024.
15. Accept a sentence from a user and capitalize the first letter of each word.
16. Find the sum of each row of matrix of size m x n.

Introduction to Tuples

- Sequence of immutable Python objects
- Enclosed within ( )
- The elements can be of any data type
- Almost similar to a list but is immutable
- Examples:

```
>>> t1 = (1, 2, 56, 87)
>>> t2 = ('physics', 'chemistry', 127, 456)
>>> t3 = (40,)
>>> t4 = 2, 4, 5
```

Creating Tuples and Accessing Values

- Creating an empty tuple:

```
>>> t = ()
```

- Creating a tuple with a single element:

```
>>> t = (45,)      # Without a comma, it would be considered an integer
```

- Accessing values in a tuple:

```
>>> t = (1, 2, 3, 4, 5, 6)
>>> t[0]
1
>>> t[2:5]
(3, 4, 5)
```

Modifying a Tuple

- It is not possible to add, modify, or delete an item from a tuple.
- However, a mutable object inside a tuple, such as a list or a dictionary, can be modified.
- Example:

```
>>> t = (1, [2,3], 4)
>>> t[1][0] = 4
>>> t
(1, [4, 3], 4)
>>> t[1] = 5
Error!
```

Operations / Methods

- Concatenation

```
>>> (1, 2, 3) + ('a', 'f', 'x')
(1, 2, 3, 'a', 'f', 'x')
```

- Repetition

```
>>> (1, 2) * 3
(1, 2, 1, 2, 1, 2)
```

- Sorting

```
>>> t = (1, 6, 4, 8, 0)
>>> sorted(t)          # Results in a list
[0, 1, 4, 6, 8]
```

More Operations / Methods

- Finding the index of an element using *index*

```
>>> t = (1, 2, 3, 4, 4, 3)
>>> t.index(3)          # Returns index of the first appearance of 3
2
>>> t.index(3,2)        # Returns index + 2
4
```

- Finding the number of times an element occurs in a tuple using *count*

```
>>> t.count(3)
2
```

More Operations / Methods

- Individual elements of a tuple cannot be deleted but the tuple as a whole can be deleted using ***del***.

```
>>> t = (1, 2, 3)
>>> del t
```

- Length of a tuple using ***len***
- Conversion to tuple using ***tuple***
- Functions such as ***sum, max, min*** may all be used on tuples

```
>>> max(t)
3
```

Tuple as Return Value

- Functions return multiple values using tuples.
- Example:

$def\ func(a, b):$

$\quad c = a + b$

$\quad d = a - b$

$\quad e = a * b$

$\quad return\ c, d, e$

$print(func(3,2))$

$>>> (5, 1, 6)$

Programming Exercises

1. Compute the sum of all the elements of each tuple stored inside a list of tuples.

   Example: Input: [(1, 2), (2, 3), (3, 4)]

   Output: [3, 5, 7]

2. Obtain the percentage mark of N students and store them in a list of tuples, where each tuple contains the name and the mark. Display the tuple containing the k-th highest mark.

3. Write a function that takes a string as an input and returns the most frequently occurring character and its count as a tuple.

4. Given a list of distinct positive integers, return the tuples (a, b, c, d) such that $a * b = c * d$ where a, b, c, and d are elements of the list, and $a \neq b \neq c \neq d$